

Application Note

Migrating Software From 8-Bit (Byte) Addressable CPUs to C28x CPU



Veena Kamath

ABSTRACT

While developing applications, you may assume that the device runs on a 8-bit addressable device and can face issues while porting the code to a 16-bit addressable device. This application note discusses such common scenarios and provides a guide on how to develop application irrespective of the addressability.

Table of Contents

1 Introduction.....	2
2 Byte vs Word Terminology.....	2
3 Key Points to Consider.....	3
3.1 8-Bit Data Types are not Supported.....	3
3.2 Memory Size is Expressed in 16 Bits.....	4
3.3 Arrays and Structures: Individual Element Offsets are Different.....	4
3.4 Difference in Standard Data Type Widths.....	6
3.5 Dealing With 8-Bit Communication Protocols.....	6
4 References.....	6
5 Revision History.....	7

List of Figures

Figure 3-1. Memory Allocation in C28x Core.....	5
Figure 3-2. Memory Allocation in Arm Core.....	5

List of Tables

Table 2-1. Memory Map Table.....	2
----------------------------------	---

Trademarks

TMS320™ and Code Composer Studio™ are trademarks of Texas Instruments.
Arm® is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.
All trademarks are the property of their respective owners.

应用笔记

从8位（字节）寻址CPU迁移软件到C28x CPU



Veena Kamath

摘要

在开发应用程序时，您可能假设设备运行在8位寻址设备上，并且在将代码移植到16位寻址设备时可能会遇到问题。本应用笔记讨论了此类常见场景，并提供了如何在寻址方式无关的情况下开发应用程序的指南。

目录

1 简介.....	2
2 字节与术语.....	2
3 需要注意的关键点.....	3
3.1 不支持 8 位数据类型.....	3
3.2 内存大小以 16 位表示.....	4
3.3 数组和结构：单个元素偏移不同.....	4
3.4 标准数据类型宽度差异.....	6
3.5 处理 8 位通信协议.....	6
4 引用.....	6
5 版本历史.....	7

图表列表

图3-1。C28x内核的内存分配.....	5
图3-2。Arm内核的内存分配.....	5

表格列表

表2-1。内存映射表.....	2
-----------------	---

商标

TMS320™ 和Code Composer Studio™ 是德州仪器的商标。
Arm® 是Arm Limited（或其子公司）在美国和/或其他地区的注册商标。
所有商标都是其各自所有者的财产。

1 Introduction

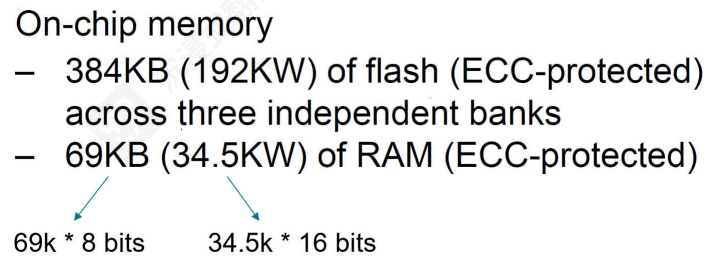
The TMS320C28x is one of several fixed-point CPUs in the TMS320™ family. It has a 16-bit addressable architecture compared to 8-bit addressable architecture like Arm® CPUs.

In a 8-bit addressable architecture, every 8 bits of data has a unique address, where as in a 16-bit addressable device , every 16 bits of data has a unique address. Hence, in a 16-bit architecture, the smallest addressable unit of memory and the smallest data type supported is 16 bits.

2 Byte vs Word Terminology

Historically, byte is defined as the smallest addressable unit of memory. Hence, technically, the size of a byte is hardware dependent: 16 bits for C28x devices and 8 bits for Arm devices. But nowadays, byte is used as a synonym of 8 bits, since majority of the device architectures are 8 bit addressable. To avoid the confusion, let us use the terminology 8-bit bytes and 16-bit words

Byte and Word Usage in the device feature set Documentation:



From the C28x compiler's perspective, the memory size is always expressed in smallest addressable units, which is 16 bits. This includes the memory length, code/data size provided in the linker cmd files, .map file, and so forth. The standard sizeof() function also returns the size in 16 bits.

- Memory Map Table in device-specific data sheet:

Table 2-1. Memory Map Table

Memory	Size	Start Address	End Address
LS0 RAM	2K × 16	0x0000 8000	0x0000 87FF
LS1 RAM	2K × 16	0x0000 8800	0x0000 8FFF

- Linker command file:

```
RAMLS0    : origin = 0x00008000, length = 0x0800
RAMLS1    : origin = 0x00008800, length = 0x0800
```

- .map file generated by Code Composer Studio™ (CCS):

MEMORY CONFIGURATIONS					
name	origin	length	used	unused	attr
-----	-----	-----	----	-----	----
RAMLS0	00008000	00000800	00000112	000006ee	RWIX
RAMLS1	00008800	00000800	00000194	0000066c	RWIX

- sizeof(uint32_t) = 2 → 2 * 16 bits

1 简介

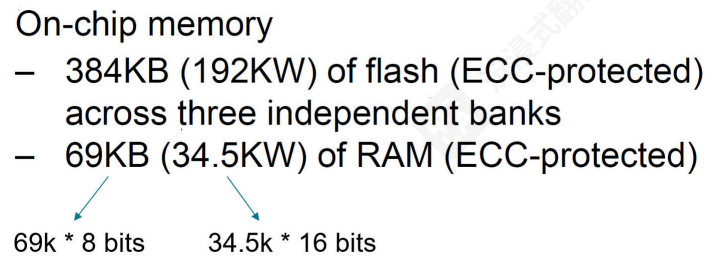
TMS320C28x 是 TMS320™ 系列中几种定点 CPU 之一。它与 Arm® CPU 等具有 8 位可寻址架构相比，具有 16 位可寻址架构。

在8位可寻址架构中，每8位数据都有一个唯一的地址，而在16位可寻址设备中，每16位数据都有一个唯一的地址。因此，在16位架构中，内存的最小可寻址单元和所支持的最小数据类型都是16位。

2 字节与字术语

历史上，字节被定义为内存的最小可寻址单元。因此，从技术上讲，字节的大小是硬件相关的：C28x设备为16位，Arm设备为8位。但如今，字节被用作8位的同义词，因为大多数设备架构都是8位可寻址的。为了避免混淆，让我们使用8位字节和16位字的术语

设备特性集文档中的字节和字使用情况：



从C28x编译器的角度来看，内存大小始终以最小的可寻址单元表示，即16位。这包括链接器cmd文件中提供的内存长度、代码/数据大小、.map文件等。标准的sizeof()函数也返回16位的大小。

- 设备特定数据表中的内存映射表：

表 2-1. 内存映射表

内存	Size	起始地址	结束地址
LS0 RAM	2K × 16	0x0000 8000	0x0000 87FF
LS1 RAM	2K × 16	0x0000 8800	0x0000 8FFF

- 链接器命令文件：

```
RAMLS0    : origin = 0x00008000, length = 0x0800
RAMLS1    : origin = 0x00008800, length = 0x0800
```

- .map 文件由 Code Composer Studio™ (CCS) 生成：

MEMORY CONFIGURATIONS					
name	origin	length	used	unused	attr
-----	-----	-----	----	-----	----
RAMLS0	00008000	00000800	00000112	000006ee	RWIX
RAMLS1	00008800	00000800	00000194	0000066c	RWIX

- sizeof(uint32_t) = 2 → 2 * 16 位

3 Key Points to Consider

3.1 8-Bit Data Types are not Supported

In C28x CPU-based projects, 8-bit data types are not supported. char is 16 bits wide and the types uint8_t and int8_t are not defined by the C28x compiler. C2000Ware remaps these to uint16_t and int16_t data types. For more details on the data types, see the [TMS320C28x Optimizing C/C++ Compiler v22.6.0.LTS User's Guide](#).

However, the C28x compiler provides intrinsic __byte() for byte accesses. For more details, see the https://software-dl.ti.com/ccs/esd/documents/c2000_byte-accesses-with-the-c28x-cpu.html.

- While porting an application from an 8-bit addressable architecture, such as Arm to C28x, you may find an increase in memory requirement due to this difference.

Example:

```
struct
{
    uint8_t a;
    uint8_t b;
    uint16_t c;
} myStruct;
```

In an Arm device, the size of myStruct would be 8 + 8 + 16 = 32 bits. Whereas, in C28x, the size would be 16 + 16 + 16 = 48 bits.

- Since the int8 or char data types in C28x is 16 bit wide, the compiler will not do a wraparound at 0xFF while performing arithmetic or shift operations.

Example:

```
uint8_t a = 0xFF;
a += 1;
if (a == 0)
{
    //Condition is true for Arm and false for C28x
}
```

- While converting larger data types to smaller ones or vice versa, care should be taken to properly typecast, use masks, and be aware of the device endianness. The C28x-based devices are little endian devices.

uint8_t a[4] = {0x1, 0x2, 0x3, 0x4};

uint32_t b1 = *(uint32_t *)a; ----- ✗ The result varies with endianness as well. C28x is little endian device and b1 = 0x00020001. In ARM little endian device, b1 = 0x04030201

uint32_t b2 = (a[0] << 0) | (a[1] << 8) | (a[2] << 16) | (a[3] << 24); ----- ✗ Shifting beyond the data type size. The behavior is compiler specific. In the C28x compiler, the value is truncated if shifted beyond its size. It also throws a warning

uint32_t b2 = ((uint32_t)a[0] << 0) | ((uint32_t)a[1] << 8) | ((uint32_t)a[2] << 16) | ((uint32_t)a[3] << 24); ----- ✓ Recommended usage of typecasts and shift operators. b3 = 0x04030201 in both C28x and ARM-LE

uint32_t a = 0x12345678;

uint8_t b1 = a; ----- ✗ b1 = 0x5678 in C28x, 0x78 in ARM

uint8_t b2 = (uint8_t)a; ----- ✗ b2 = 0x5678 in C28x, 0x78 in ARM

uint8_t b3 = a & 0xFF; ----- ✓ Recommended usage of masks. b3 = 0x78 in both C28x and ARM

3 个需要考虑的关键点

3.1 8 位数据类型不受支持

在基于 C28x CPU 的项目中，不支持 8 位数据类型。char 是 16 位宽，uint8_t 和 int8_t 类型没有被 C28x 编译器定义。C2000Ware 将这些重新映射为 uint16_t 和 int16_t 数据类型。有关更多关于数据类型的详细信息，请参阅 TMS320C28x 优化 C/C++ 编译器 v22.6.0.LTS 用户指南。

然而，C28x 编译器提供了内置的 __byte() 用于字节访问。更多详情，请参阅 https://software-dl.ti.com/ccs/esd/documents/c2000_byte-accesses-with-the-c28x-cpu.html。

- 在将应用程序从 8 位可寻址架构（例如 Arm）移植到 C28x 时，由于这种差异，您可能会发现内存需求增加。

示例：

```
struct
{
    uint8_t a;
    uint8_t b;
    uint16_t c;
} myStruct;
```

在 Arm 设备中，myStruct 的大小将是

8 + 8 + 16 = 32

28

16 位。而在 C x 中，大小将是位。

- 由于 C28x 中的 int8 或 char 数据类型是 16 位宽，编译器在进行算术或移位操作时不会在 0xFF 处进行回绕。

示例：

```
uint8_t a = 0xFF; a += 1; if (a == 0) { //对于 Arm 为真，对于 C28x 为假 }
```

- 在将较大的数据类型转换为较小的类型或反之亦然时，应注意正确进行类型转换、使用掩码，并了解设备端序。基于 C28x 的设备是小端设备。

uint8_t a[4] = {0x1, 0x2, 0x3, 0x4};

uint32_t b1 = *(uint32_t *)a; ----- ✗ The result varies with endianness as well. C28x is little endian device and b1 = 0x00020001. In ARM little endian device, b1 = 0x04030201

uint32_t b2 = (a[0] << 0) | (a[1] << 8) | (a[2] << 16) | (a[3] << 24); ----- ✗ Shifting beyond the data type size. The behavior is compiler specific. In the C28x compiler, the value is truncated if shifted beyond its size. It also throws a warning

uint32_t b2 = ((uint32_t)a[0] << 0) | ((uint32_t)a[1] << 8) | ((uint32_t)a[2] << 16) | ((uint32_t)a[3] << 24); ----- ✓ Recommended usage of typecasts and shift operators. b3 = 0x04030201 in both C28x and ARM-LE

uint32_t a = 0x12345678;

uint8_t b1 = a; ----- ✗ b1 = 0x5678 in C28x, 0x78 in ARM

uint8_t b2 = (uint8_t)a; ----- ✗ b2 = 0x5678 in C28x, 0x78 in ARM

uint8_t b3 = a & 0xFF; ----- ✓ Recommended usage of masks. b3 = 0x78 in both C28x and ARM

- Usage of unions needs to be revisited.

```
union
{
    struct
    {
        uint8_t byte1;
        uint8_t byte2;
        uint8_t byte3;
        uint8_t byte4;
    };
    uint32_t word;
}test;

union
{
    struct
    {
        uint8_t byte1 : 8;
        uint8_t byte2 : 8;
        uint8_t byte3 : 8;
        uint8_t byte4 : 8;
    };
    uint32_t word;
}test;
```

The total size of the union will be 64 bits in C28x and 32 bits in ARM
✗ b1=1, b2=1, b3 =1, b4 =1 → word = 0x00010001 in C28x
→ word = 0x01010101 in ARM

✓ Recommended usage of bitfields
b1=1, b2=1, b3 =1, b4 =1 → word = 0x01010101 in C28x and ARM

3.2 Memory Size is Expressed in 16 Bits

The memory sizes mentioned in the linker cmd files, .map files are all expressed in 16 bits. The sizeof() function always returns the size with respect to smallest addressable memory units (8 bits for Arm and 16 bits for C28x).

Be careful to not hardcode the size information in the application. One common scenario is while using memset/cpy/cmp functions, the size parameter to these functions are in terms of smallest addressable units.

```
struct
{
    uint16_t a;
    uint16_t b;
    uint32_t c;
} myStruct;
```

The total size of the struct is 64 bits in C28x and in ARM.
But sizeof(myStruct) = 4 in C28x, and 8 in ARM

✗ Hardcoded memory size – In the case of C28x, corrupts the adjacent memory
memset(&myStruct, 0xA, 8);

✓ Recommended usage of sizeof() instead of assuming it as 8
memset(&myStruct, 0xA, sizeof(myStruct));

Also, note the difference in data packing after memset usage:

C28x	Arm
myStruct 000A000A 000A000A	myStruct 0A0A0A0A 0A0A0A0A

3.3 Arrays and Structures: Individual Element Offsets are Different

- In C28x, there is a unique address for every 16 bits, as opposed to 8 bits in an Arm device.

Example:

```
uint32_t Array32[4] = {1,2,3,4};
uint16_t Array16[4] = {1,2,3,4};
uint8_t Array8[4] = {1,2,3,4};
```

- 需要重新审视联合的使用。

```
union
{
    struct
    {
        uint8_t byte1;
        uint8_t byte2;
        uint8_t byte3;
        uint8_t byte4;
    };
    uint32_t word;
}test;

union
{
    struct
    {
        uint8_t byte1 : 8;
        uint8_t byte2 : 8;
        uint8_t byte3 : 8;
        uint8_t byte4 : 8;
    };
    uint32_t word;
}test;
```

The total size of the union will be 64 bits in C28x and 32 bits in ARM
✗ b1=1, b2=1, b3 =1, b4 =1 → word = 0x00010001 in C28x
→ word = 0x01010101 in ARM

✓ Recommended usage of bitfields
b1=1, b2=1, b3 =1, b4 =1 → word = 0x01010101 in C28x and ARM

3.2 内存大小以16位表示

链接器cmd文件中提到的内存大小，.map文件中的内存大小都以16位表示。sizeof()函数始终返回相对于最小可寻址内存单元的大小（ARM为8位，C28x为16位）。

注意不要在应用程序中硬编码大小信息。一个常见场景是使用memset/cpy/cmp函数时，这些函数的大小参数以最小可寻址单元表示。

```
struct
{
    uint16_t a;
    uint16_t b;
    uint32_t c;
} myStruct;
```

The total size of the struct is 64 bits in C28x and in ARM.
But sizeof(myStruct) = 4 in C28x, and 8 in ARM

✗ Hardcoded memory size – In the case of C28x, corrupts the adjacent memory
memset(&myStruct, 0xA, 8);

✓ Recommended usage of sizeof() instead of assuming it as 8
memset(&myStruct, 0xA, sizeof(myStruct));

此外，注意memset使用后的数据打包差异：

C28x	Arm
myStruct 000A000A 000A000A	myStruct 0A0A0A0A 0A0A0A0A

3.3 数组和结构：单个元素偏移不同

- 在C28x中，每个16位都有唯一的地址，而Arm设备中是8位。

示例：

```
uint32_t Array32[4] = {1,2,3,4}; uint16_t Array16[4] = {1,2,3,4}; uint8_t Array8[4] = {1,2,3,4};
```


Expression	Type	Value	Address	
Array32	unsigned long[4]	[1,2,3,4]	0x0000A800@Data	
(x)= [0]	unsigned long	1	0x0000A800@Data	Address increments by 2 for a 32-bit array
(x)= [1]	unsigned long	2	0x0000A802@Data	
(x)= [2]	unsigned long	3	0x0000A804@Data	
(x)= [3]	unsigned long	4	0x0000A806@Data	
Array16	unsigned int[4]	[1,2,3,4]	0x0000A80E@Data	
(x)= [0]	unsigned int	1	0x0000A80E@Data	Address increments by 1 for a 16-bit array
(x)= [1]	unsigned int	2	0x0000A80F@Data	
(x)= [2]	unsigned int	3	0x0000A810@Data	
(x)= [3]	unsigned int	4	0x0000A811@Data	
Array8	unsigned int[4]	[1,2,3,4]	0x0000A812@Data	
(x)= [0]	unsigned int	1	0x0000A812@Data	uint8_t = uint16_t Address increments by 1
(x)= [1]	unsigned int	2	0x0000A813@Data	
(x)= [2]	unsigned int	3	0x0000A814@Data	
(x)= [3]	unsigned int	4	0x0000A815@Data	

Figure 3-1. Memory Allocation in C28x Core

Expression	Type	Value	Address	
Array32	unsigned long[4]	[1,2,3,4]	0x0000A800@Data	
(x)= [0]	unsigned long	1	0x0000A800@Data	Address increments by 2 for a 32-bit array
(x)= [1]	unsigned long	2	0x0000A802@Data	
(x)= [2]	unsigned long	3	0x0000A804@Data	
(x)= [3]	unsigned long	4	0x0000A806@Data	
Array16	unsigned int[4]	[1,2,3,4]	0x0000A80E@Data	
(x)= [0]	unsigned int	1	0x0000A80E@Data	Address increments by 1 for a 16-bit array
(x)= [1]	unsigned int	2	0x0000A80F@Data	
(x)= [2]	unsigned int	3	0x0000A810@Data	
(x)= [3]	unsigned int	4	0x0000A811@Data	
Array8	unsigned int[4]	[1,2,3,4]	0x0000A812@Data	
(x)= [0]	unsigned int	1	0x0000A812@Data	uint8_t = uint16_t Address increments by 1
(x)= [1]	unsigned int	2	0x0000A813@Data	
(x)= [2]	unsigned int	3	0x0000A814@Data	
(x)= [3]	unsigned int	4	0x0000A815@Data	

图 3-1. C28x 内核的内存分配

Expression	Type	Value	Address	
Array32	unsigned int[4]	[1,2,3,4]	0x2000C000	
(x)= [0]	unsigned int	1	0x2000C000	Address increments by 4 for a 32-bit array
(x)= [1]	unsigned int	2	0x2000C004	
(x)= [2]	unsigned int	3	0x2000C008	
(x)= [3]	unsigned int	4	0x2000C00C	
Array16	unsigned short[4]	[1,2,3,4]	0x2000C010	
(x)= [0]	unsigned short	1	0x2000C010	Address increments by 2 for a 16-bit array
(x)= [1]	unsigned short	2	0x2000C012	
(x)= [2]	unsigned short	3	0x2000C014	
(x)= [3]	unsigned short	4	0x2000C016	
Array8	unsigned char[4]	[1 '\x01',2 '...']	0x2000C018	
(x)= [0]	unsigned char	1 '\x01'	0x2000C018	Address increments by 1 for a 8-bit array
(x)= [1]	unsigned char	2 '\x02'	0x2000C019	
(x)= [2]	unsigned char	3 '\x03'	0x2000C01A	
(x)= [3]	unsigned char	4 '\x04'	0x2000C01B	

Figure 3-2. Memory Allocation in Arm Core

Expression	Type	Value	Address	
Array32	unsigned int[4]	[1,2,3,4]	0x2000C000	
(x)= [0]	unsigned int	1	0x2000C000	Address increments by 4 for a 32-bit array
(x)= [1]	unsigned int	2	0x2000C004	
(x)= [2]	unsigned int	3	0x2000C008	
(x)= [3]	unsigned int	4	0x2000C00C	
Array16	unsigned short[4]	[1,2,3,4]	0x2000C010	
(x)= [0]	unsigned short	1	0x2000C010	Address increments by 2 for a 16-bit array
(x)= [1]	unsigned short	2	0x2000C012	
(x)= [2]	unsigned short	3	0x2000C014	
(x)= [3]	unsigned short	4	0x2000C016	
Array8	unsigned char[4]	[1 '\x01',2 '...']	0x2000C018	
(x)= [0]	unsigned char	1 '\x01'	0x2000C018	Address increments by 1 for a 8-bit array
(x)= [1]	unsigned char	2 '\x02'	0x2000C019	
(x)= [2]	unsigned char	3 '\x03'	0x2000C01A	
(x)= [3]	unsigned char	4 '\x04'	0x2000C01B	

图 3-2. Arm 内核的内存分配

- While using arrays, the index-based accesses and pointer increment-based access would yield the same results in both the devices. But if you try to use hardcoded offsets, it would produce different results.

```
uint32_t read1 = Array32[2]; ..... ✓ Index-based access - compiler takes care of using the correct offset
uint32_t read2 = *(Array32 + 2); ..... ✓ Pointer increment – compiler takes care of incrementing by the right size
uint32_t read3 = *((uint32_t *)(baseAddr + 8)); ..... ✗ Hardcoded offset value – Gives different results in C28x and ARM
uint32_t read4 = *((uint32_t *)(baseAddr + 2*sizeof(uint32_t))); ..... ✓ Recommended usage of sizeof() instead of assuming it as 4
```

- Same is applicable for structs. While struct.element provides the same results in both devices, the usage of (baseAddr + offset) to access struct.

- 在使用数组时，基于索引的访问和基于指针递增的访问在两种设备上都会产生相同的结果。但如果您尝试使用硬编码的偏移量，它会产生不同的结果。

```
uint32_t read1 = Array32[2]; ..... ✓ Index-based access - compiler takes care of using the correct offset
uint32_t read2 = *(Array32 + 2); ..... ✓ Pointer increment – compiler takes care of incrementing by the right size
uint32_t read3 = *((uint32_t *)(baseAddr + 8)); ..... ✗ Hardcoded offset value – Gives different results in C28x and ARM
uint32_t read4 = *((uint32_t *)(baseAddr + 2*sizeof(uint32_t))); ..... ✓ Recommended usage of sizeof() instead of assuming it as 4
```

- 结构体也适用。虽然 struct.element 在两种设备上都能提供相同的结果，但使用 (baseAddr + offset) 来访问结构体。

3.4 Difference in Standard Data Type Widths

As opposed to any 8-bit addressable architecture, the sizes of int and char are different in C28x devices. For better portability, it is highly recommended to use the width-based data types such as uint16_t, int16_t, uint32_t, int32_t, uint64_t, int64_t, float32_t, float64_t, and so forth. These data types are defined in the C28 compiler header file stdint.h.

Note that the data types uint8_t and int8_t are not defined by the C28x compiler. C2000Ware remaps these to uint16_t and int16_t data types, respectively.

Type	Size
char	16 bits
_Bool	16 bits
short	16 bits
int	16 bits
long	32 bits
long long	64 bits
float	32 bits
double(COFF)	32 bits
double(EABI)	64 bits
long double	64 bits
Pointers	32 bits

For more details on the data types, see the [TMS320C28x Optimizing C/C++ Compiler v22.6.0.LTS User's Guide](#).

3.5 Dealing With 8-Bit Communication Protocols

Communication peripherals like the Controller Area Network (CAN), Serial Communications Interface (SCI) and Universal Asynchronous Receiver/Transmitter (UART) in C28x devices support 8-bit data transfers. But note that the data would be stored as 16 bits in the memory. The C28x compiler does not support pragma pack(1).

Example:

```
unsigned char *msg;  
msg = "\r\nEnter a character: \0";  
SCI_writeCharArray(SCIA_BASE, (uint16_t*)msg, 22);
```



Memory Browser view
1 character = 16 bits



The SCI TX pin captured in a
logic analyzer.
1 character = 8 bits

In this example, the function takes in uint16_t * as the input data parameter. But it only expects 8 bits of data per uint16_t type. The upper half of the uint16_t is ignored and only the lower 8 bits are transferred.

4 References

- [TMS320C28x Optimizing C/C++ Compiler v22.6.0.LTS User's Guide](#)
- https://software-dl.ti.com/ccs/esd/documents/c2000_byte-accesses-with-the-c28x-cpu.html

3.4 标准数据类型宽度差异

与任何8位可寻址架构不同，在C28x设备中，int 和 char 的大小是不同的。为了更好的可移植性，强烈建议使用基于位宽的数据类型，例如 uint16_t、int16_t、uint32_t、int32_t、uint64_t、int64_t、float32_t、float64_t 等等。这些数据类型在C28编译器头文件 stdint.h 中定义。

请注意，C28x 编译器未定义 uint8_t 和 int8_t 数据类型。C2000Ware 将这些分别重新映射为 uint16_t 和 int16_t 数据类型。

Type	Size
char	16 位
_Bool	16 位
short	16 位
int	16 位
long	32 位
long long	64 位
float	32 位
double(COFF)	32 位
double(EABI)	64 位
long double	64 位
指针	32 位

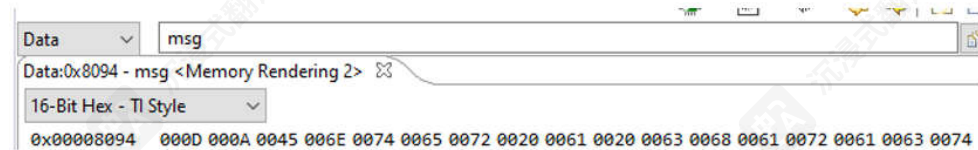
有关数据类型的更多详细信息，请参阅 TMS320C28x 优化 C/C++ 编译器 v22.6.0.LTS 用户指南。

3.5 处理 8 位通信协议

通信外设，如 C28x 设备中的控制器局域网 (CAN)、串行通信接口 (SCI) 和通用异步收发器 (UART) 支持 8 位数据传输。但请注意，数据在内存中会以 16 位的形式存储。C28x 编译器不支持 pragma pack(1)。

示例：

```
unsigned char *msg;  
msg = "\r\nEnter a character: \0";  
SCI_writeCharArray(SCIA_BASE, (uint16_t*)msg, 22);
```



Memory Browser view
1 character = 16 bits



The SCI TX pin captured in a
logic analyzer.
1 character = 8 bits

在这个示例中，函数以 uint16_t * 作为输入数据参数。但它只期望每个 uint16_t 类型 8 位数据。uint16_t 的高位被忽略，只有低 8 位被传输。

4 个参考文献

- [TMS320C28x 优化 C/C++ 编译器 v22.6.0.LTS 用户指南](#)
- https://software-dl.ti.com/ccs/esd/documents/c2000_byte-accesses-with-the-c28x-cpu.html

5 Revision History

NOTE: Page numbers for previous revisions may differ from page numbers in the current version.

Changes from Revision * (March 2023) to Revision A (April 2023)	Page
• Updated the numbering format for tables, figures and cross-references throughout the document.....	2
• Updated Section 2	2
• Updated Section 3.1	3
• Updated Section 3.2	4

5 版本修订历史 注意：先前版本的页码可能与当前版本的页码不同。修订 *（2023年3月）* 至修
订 A（2023年4月）的变更

	Page
• 更新了文档中表格、图表和交叉引用的编号格式.....	2
• 更新 d Sectio n 2	2
• 更新 d Sectio n 3:1	3
• 更新 d Sectio n 3.2	4

IMPORTANT NOTICE AND DISCLAIMER

TI PROVIDES TECHNICAL AND RELIABILITY DATA (INCLUDING DATA SHEETS), DESIGN RESOURCES (INCLUDING REFERENCE DESIGNS), APPLICATION OR OTHER DESIGN ADVICE, WEB TOOLS, SAFETY INFORMATION, AND OTHER RESOURCES “AS IS” AND WITH ALL FAULTS, AND DISCLAIMS ALL WARRANTIES, EXPRESS AND IMPLIED, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS.

These resources are intended for skilled developers designing with TI products. You are solely responsible for (1) selecting the appropriate TI products for your application, (2) designing, validating and testing your application, and (3) ensuring your application meets applicable standards, and any other safety, security, regulatory or other requirements.

These resources are subject to change without notice. TI grants you permission to use these resources only for development of an application that uses the TI products described in the resource. Other reproduction and display of these resources is prohibited. No license is granted to any other TI intellectual property right or to any third party intellectual property right. TI disclaims responsibility for, and you will fully indemnify TI and its representatives against, any claims, damages, costs, losses, and liabilities arising out of your use of these resources.

TI's products are provided subject to [TI's Terms of Sale](#) or other applicable terms available either on [ti.com](#) or provided in conjunction with such TI products. TI's provision of these resources does not expand or otherwise alter TI's applicable warranties or warranty disclaimers for TI products.

TI objects to and rejects any additional or different terms you may have proposed.

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2024, Texas Instruments Incorporated

重要通知和免责声明

TI 提供技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、网络工具、安全信息和其他资源“按原样”并带有所有缺陷，并放弃所有明示和默示的保证，包括但不限于任何默示的适销性保证、特定用途适用性保证或第三方知识产权非侵权保证。

这些资源旨在供使用 TI 产品的熟练开发者使用。您仅对（1）为您的应用选择合适的 TI 产品、（2）设计、验证和测试您的应用以及（3）确保您的应用符合适用标准以及任何其他安全、安全、监管或其他要求负全部责任。

这些资源如有更改恕不另行通知。TI 仅授权您使用这些资源开发使用资源中所述的 TI 产品的应用。禁止其他形式的复制和展示这些资源。不授予任何其他 TI 知识产权或任何第三方知识产权的许可。TI 不对您使用这些资源产生的任何索赔、损害赔偿、费用、损失和责任承担责任，您将充分赔偿 TI 及其代表因您使用这些资源而产生的任何索赔、损害赔偿、费用、损失和责任。

TI 的产品根据 TI 的销售条款或其他在 ti.com 上提供或随此类 TI 产品一同提供的适用条款提供。TI 提供这些资源并不扩大或以其他方式改变 TI 产品的适用保修或保修免责声明。

TI 反对并拒绝您可能提出的任何附加或不同的条款。

邮寄地址：德州仪器公司，邮政信箱 655303，德克萨斯州达拉斯，75265 版权所有 ©
2024，德州仪器公司